OPTIMIZATION FOR MACHINE LEARNING

Smarter gradient descent

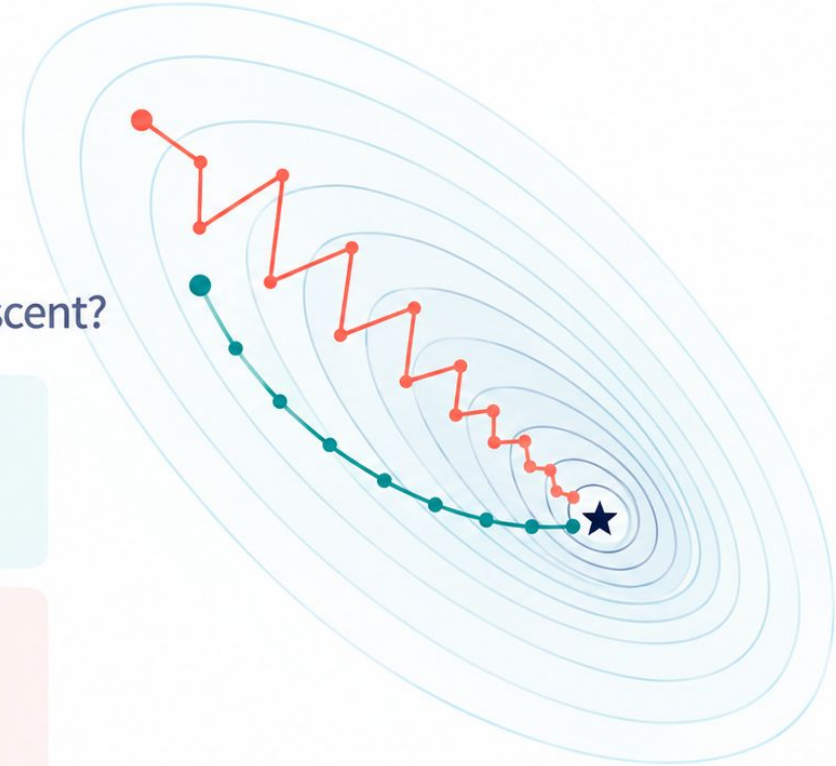
How can we improve stochastic gradient descent?

Build momentum

Use gradient history to smooth updates and accelerate progress.

Adapt the learning rate

Scale updates using gradient history, with a separate effective rate for each parameter.



Illustrative optimization paths

Momentum → **AdaGrad** → **RMSprop**

Gradient update methods

Four ways to adapt parameter updates

Method	What it tracks	How it scales the update
AdaGrad	Sum of all past squared gradients	Divide the current gradient by the square root of the accumulated sum.
RMSprop	Exponential average of squared gradients	Divide the current gradient by a recent gradient scale.
AdaDelta	Exponential averages of squared gradients and squared updates	Scale the current gradient by the ratio of past update RMS to current gradient RMS.
Adam	Exponential averages of gradients and squared gradients	Use the bias-corrected gradient average, scaled by the corrected squared-gradient average's square root.

RMS uses exponential averages here; AdaGrad uses a cumulative sum. Small ε terms provide numerical stability.

Further reading: Sebastian Ruder · An overview of gradient descent optimization algorithms · arXiv:1609.04747

AdaGrad

Adaptive step sizes from gradient history

Standard gradient descent

$$\theta_{t+1,i} = \theta_{t,i} - \eta\, g_{t,i}$$

AdaGrad update

Accumulated squared gradients: $$G_{t,i} = \sum_{\tau=1}^{t} g_{\tau,i}^2$$

Parameter update: $$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \varepsilon}\, g_{t,i}$$

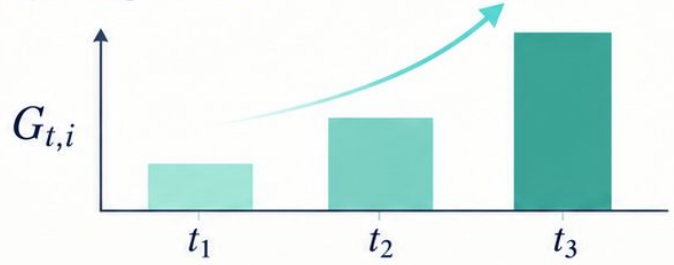
Definitions:

$$g_{t,i} = \left.\frac{\partial C(\theta)}{\partial \theta_i}\right|_{\theta=\theta_t}$$

- η Learning rate
- ε Numerical stability

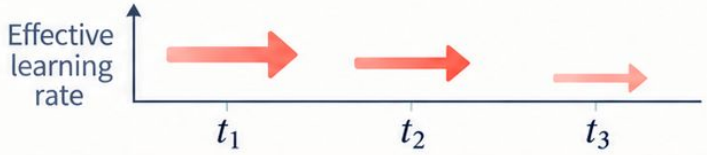
Gradient history grows

Each parameter accumulates the sum of past squared gradients.



Effective learning rate decreases

With a constant η, the effective learning rate decreases as accumulated squared gradients grow.



KEY TAKEAWAY

Larger accumulated squared gradients yield smaller effective learning rates.

RMSprop

Adapt to recent gradients with exponential averaging

Why change AdaGrad?

Accumulating all past squared gradients can make the effective learning rate too small.

RMSprop update

Exponential average of squared gradients:

$$v_{t,i} = \gamma\, v_{t-1,i} + (1-\gamma)\, g_{t,i}^2$$

Parameter update:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{v_{t,i}} + \varepsilon}\, g_{t,i}$$

$v_{0,i} = 0$ · $0 < \gamma < 1$ · All operations are per parameter.

Exponential memory

$$v_{t,i} = (1-\gamma) \sum_{\tau=1}^{t} \gamma^{t-\tau} g_{\tau,i}^2$$

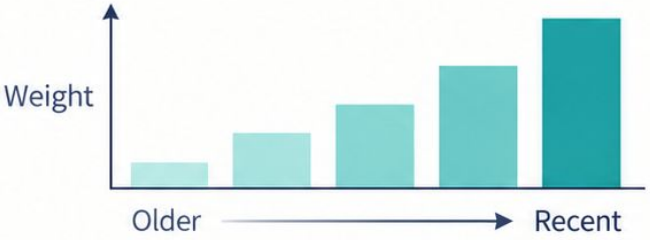
For a constant gradient g and $v_0 = 0$:

$$v_t = (1-\gamma^t)\, g^2 \;\rightarrow\; g^2 \qquad (\text{as } t \rightarrow \infty).$$

- γ Decay factor $(0 < \gamma < 1)$
- η Learning rate
- ε Numerical stability

Recent gradients matter more

Older contributions decay expomentially.



Memory scale

The expomential average effectively remembers about

$$\approx \frac{1}{1-\gamma} \quad \text{iterations.}$$

$\gamma = 0.9 \;\rightarrow\;$ about 10 iterations.

A soft memory scale, not a fixed window.

KEY TAKEAWAY

Track recent gradient scale instead of accumulating forever.

RMSprop

Scale each update by recent gradient magnitude

Parameter update

$$\Delta\theta_{t,i} = -\eta \frac{g_{t,i}}{\sqrt{v_{t,i} + \varepsilon}}$$

$$\theta_{t+1,i} = \theta_{t,i} + \Delta\theta_{t,i}$$

1 Measure

Current gradient $g_{t,i}$ sets the direction.

2 Estimate

$$v_{t,i} = \gamma v_{t-1,i} + (1-\gamma) g_{t,i}^2$$

Track recent squared gradients.

3 Rescale

$$\eta_{\text{eff},t,i} = \frac{\eta}{\sqrt{v_{t,i} + \varepsilon}}$$

Apply a learning rate for each parameter.



KEY TAKEAWAY

Larger v means a smaller effective learning rate — for the same current gradient.

η: learning rate | γ: decay factor | $\varepsilon > 0$: numerical stability

Convention shown: ε inside the square root.

Adam

Momentum + adaptive scaling + bias correction

1. Track two moving averages

Compute exponential moving averages of the gradients and squared gradients.

Gradient average: $$m_{t,i} = \beta_1 m_{t-1,i} + (1-\beta_1) g_{t,i}$$

Squared-gradient average: $$v_{t,i} = \beta_2 v_{t-1,i} + (1-\beta_2) g_{t,i}^2$$

2. Correct initialization bias

The moving averages are initialized at zero, so early estimates are biased. Use bias-corrected estimates:

$$\hat{m}_{t,i} = \frac{m_{t,i}}{1-\beta_1^t} \qquad \hat{v}_{t,i} = \frac{v_{t,i}}{1-\beta_2^t}$$

Initialize $m_0 = v_0 = 0$. Early estimates are biased toward zero.

All operations are per parameter.

3. Update each parameter

Smooth the gradient, then apply adaptive scaling.

$$\theta_{t+1,i} = \theta_{t,i} - \eta \frac{\hat{m}_{t,i}}{\sqrt{\hat{v}_{t,i}} + \varepsilon}$$

Common starting values

$\beta_1 = 0.9$ (momentum for gradients)

$\beta_2 = 0.999$ (momentum for squared gradients)

$\varepsilon = 10^{-8}$ (numerical stability)

$\eta = 10^{-3}$ (learning rate)

Tune η for the task.



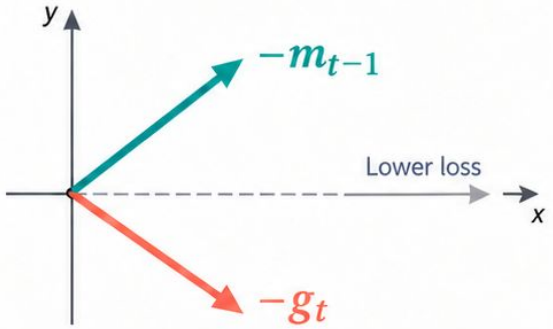
KEY TAKEAWAY

Combine a smoothed direction with per-parameter scaling.

How Adam smooths the direction

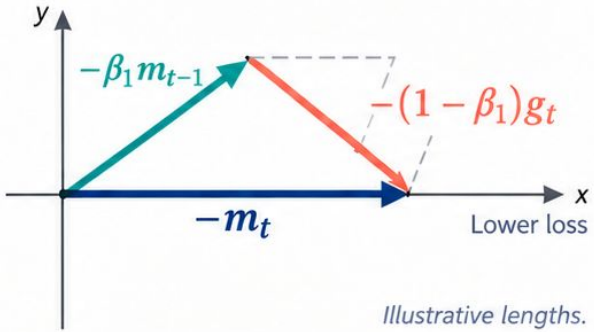
Combine gradient history with the current gradient to reduce oscillations

1. Oscillating signals



Sideways components can cancel out.

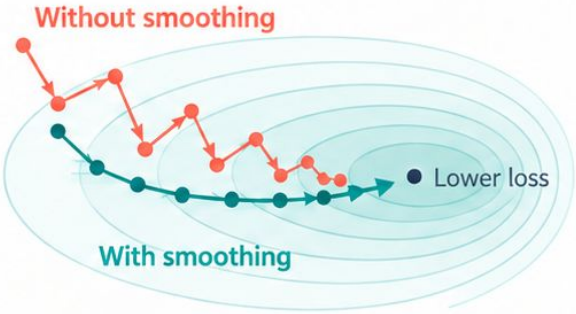
2. Weighted sum



Illustrative lengths.

Combine history with the current signal.

3. A steadier direction



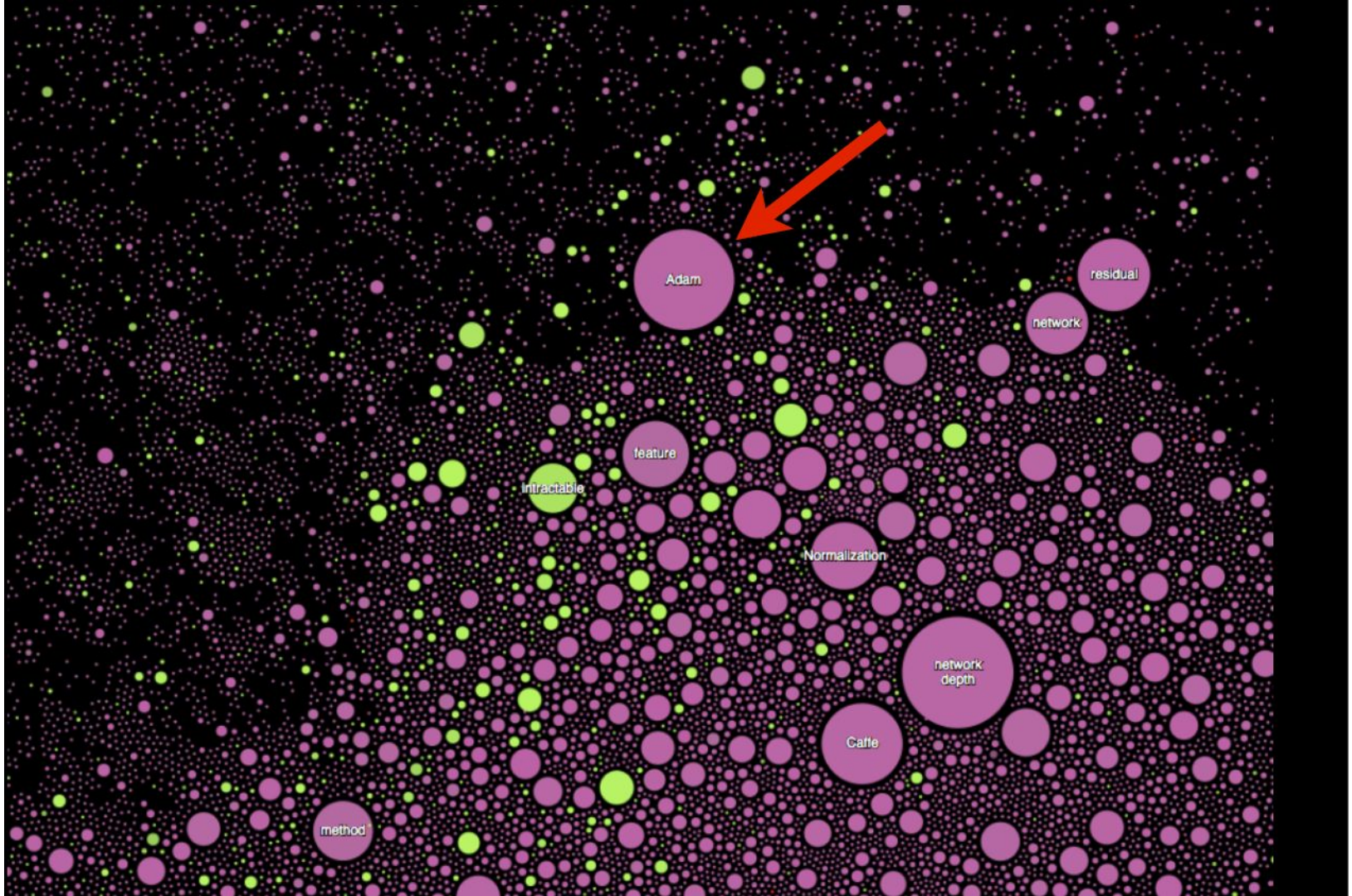
Averaging reduces abrupt direction changes.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

The smoothed descent direction is $-m_t$.

A weighted sum, not a simple sum. It does not always point toward the minimum.

In Adam, bias correction and per-parameter scaling are applied afterward.


Adam
residual
network
feature
Intractable
Normalization
network depth
Caffe
method

DEEP LEARNING BASICS LINK MAP

BASIC STRUCTURE OF NEURAL NETWORKS

Lecture 1



58 Rosenblatt

Review 89 Hinton

EXPRESS ARBITRARY FUNCTIONS WITH 1 HIDDEN LAYER

89 Cybenko

COST FUNCTION AND STOCHASTIC GRADIENT DESCENT

Lecture 2



Review 89 Hinton

BACKPROPAGATION



76 Linnainmaa

85 Rumelhart et al

IMAGE RECOGNITION

Lecture 3



DISCRETE LABELS, SOFTMAX, CATEGORICAL CROSS-ENTROPY



2021 F.M.

CONVOLUTIONAL NEURAL NETWORKS

Lecture 4



80 Fukushima

89 LeCun et al

12 Krizhevsky et al

KERNEL, CHANNELS, POOLING,...

AUTOENCODERS

Lecture 4

89 Baldi et al

06 Hinton et al

Review 12 Bengio et al



PRINCIPAL COMPONENT ANALYSIS

OTHER DIMENSIONALITY-REDUCING TECHNIQUES

Lecture 5



t-SNE

08 van der Maaten et al

RECURRENT NEURAL NETWORKS

Lecture 5

Lecture 6

LSTM



→ TIME

95 Hochreiter et al

WORD VECTORS

13 Mikolov et al

REINFORCEMENT LEARNING

Lecture 6

Lecture 7

Lecture 8



92 Williams

92 Watkins et al

BOLTZMANN MACHINE

Lecture 9



85 Sejnowski

LECTURE NOTES

Lecture Notes: 20 Marquardt

BOOKS & TUTORIALS

Book: 16 Goodfellow et al

Book: 15 Nielsen

Online course: Machine Learning for Physicists